2. Classical Synchronization Problems

5 marks

Description
1. Implement the Producer–Consumer problem using a finite circular buffer.
2. Use suitable synchronization mechanisms to control buffer access.
3. Implement the Readers–Writers problem with protected reader count and exclusive writer access.
4. Implement the Dining Philosophers problem using controlled shared-resource access.
5. Prevent permanent deadlock using resource ordering, a waiter mechanism or restricted concurrency.
6. Use finite iterations so that every program terminates normally.
7. Display the state of shared resources and thread operations.
8. Discuss deadlock, starvation, fairness and correctness.

```
import threading
import time
import random

#
=========================================================
=========
# 1. PRODUCER - CONSUMER PROBLEM
#
=========================================================
=========
```

```
print("\n" + "=" * 60)
print("1. PRODUCER - CONSUMER PROBLEM")
print("=" * 60)

BUFFER_SIZE = 5
buffer = [None] * BUFFER_SIZE

in_index = 0
out_index = 0

empty = threading.Semaphore(BUFFER_SIZE)
full = threading.Semaphore(0)
mutex = threading.Lock()


def producer():
    global in_index

    for item in range(1, 11):

        empty.acquire()

        with mutex:
            buffer[in_index] = item
            print(
                f"Producer produced: {item} | "
                f"Buffer position: {in_index}"
```

```
            )
            in_index = (in_index + 1) % BUFFER_SIZE

        full.release()
        time.sleep(0.1)


def consumer():
    global out_index

    for _ in range(10):

        full.acquire()

        with mutex:
            item = buffer[out_index]
            buffer[out_index] = None

            print(
                f"Consumer consumed: {item} | "
                f"Buffer position: {out_index}"
            )

            out_index = (out_index + 1) %
BUFFER_SIZE

        empty.release()
        time.sleep(0.15)
```

```
producer_thread = threading.Thread(target=producer)
consumer_thread = threading.Thread(target=consumer)

producer_thread.start()
consumer_thread.start()

producer_thread.join()
consumer_thread.join()

print("Producer-Consumer completed successfully.")



#
=========================================================
=========
# 2. READERS - WRITERS PROBLEM
#
=========================================================
=========

print("\n" + "=" * 60)
print("2. READERS - WRITERS PROBLEM")
print("=" * 60)

read_count = 0
shared_data = 0
```

```
read_count_lock = threading.Lock()
write_lock = threading.Lock()


def reader(reader_id):

    global read_count

    for _ in range(3):

        # Protect reader count
        with read_count_lock:
            read_count += 1

            # First reader blocks writers
            if read_count == 1:
                write_lock.acquire()

        print(
            f"Reader {reader_id} is reading "
            f"shared data = {shared_data}"
        )

        time.sleep(0.1)

        # Reader finished
        with read_count_lock:
```

```
            read_count -= 1

            # Last reader allows writers
            if read_count == 0:
                write_lock.release()

        time.sleep(0.1)


def writer(writer_id):

    global shared_data

    for _ in range(3):

        # Only one writer at a time
        with write_lock:
            shared_data += 1

            print(
                f"Writer {writer_id} is writing "
                f"shared data = {shared_data}"
            )

            time.sleep(0.2)

        time.sleep(0.1)
```

```
reader_threads = []
writer_threads = []

for i in range(3):
    t = threading.Thread(target=reader, args=(i +
1,))
    reader_threads.append(t)
    t.start()

for i in range(2):
    t = threading.Thread(target=writer, args=(i +
1,))
    writer_threads.append(t)
    t.start()


for t in reader_threads:
    t.join()

for t in writer_threads:
    t.join()

print("Readers-Writers completed successfully.")
```

```
#
================================================
=========
# 3. DINING PHILOSOPHERS PROBLEM
#
================================================
=========

print("\n" + "=" * 60)
print("3. DINING PHILOSOPHERS PROBLEM")
print("=" * 60)

NUMBER_OF_PHILOSOPHERS = 5

forks = [
    threading.Lock()
    for _ in range(NUMBER_OF_PHILOSOPHERS)
]

# Semaphore limits the number of philosophers
# trying to pick forks at the same time.
room = threading.Semaphore(NUMBER_OF_PHILOSOPHERS -
1)


def philosopher(philosopher_id):

    left_fork = philosopher_id
```

```
    right_fork = (philosopher_id + 1) %
NUMBER_OF_PHILOSOPHERS

    for meal in range(3):

        print(
            f"Philosopher {philosopher_id + 1} "
            f"is thinking."
        )

        time.sleep(random.uniform(0.1, 0.3))

        # Allow only 4 philosophers to compete
        # for forks simultaneously.
        room.acquire()

        print(
            f"Philosopher {philosopher_id + 1} "
            f"is waiting for forks."
        )

        # Acquire forks
        with forks[left_fork]:

            print(
                f"Philosopher {philosopher_id + 1}
"
                f"picked left fork."
```

```
            )

            with forks[right_fork]:

                print(
                    f"Philosopher {philosopher_id +
1} "
                    f"picked right fork and is
eating "
                    f"(Meal {meal + 1})."
                )

                time.sleep(random.uniform(0.1,
0.2))

        room.release()

        print(
            f"Philosopher {philosopher_id + 1} "
            f"finished eating."
        )


philosopher_threads = []

for i in range(NUMBER_OF_PHILOSOPHERS):

    t = threading.Thread(
```

```
        target=philosopher,
        args=(i,)
    )

    philosopher_threads.append(t)
    t.start()


for t in philosopher_threads:
    t.join()


print("\nDining Philosophers completed
successfully.")


#
==========================================================
=========
# FINAL OBSERVATION
#
==========================================================
=========

print("\n" + "=" * 60)
print("FINAL OBSERVATION")
print("=" * 60)
```

```
print("""
1. Producer-Consumer:
   - Semaphore controls empty and full buffer
slots.
   - Mutex provides exclusive buffer access.

2. Readers-Writers:
   - Multiple readers can read simultaneously.
   - Writers get exclusive access.
   - Reader count is protected by a lock.

3. Dining Philosophers:
   - Forks are shared resources.
   - Semaphore limits simultaneous philosophers.
   - This prevents permanent deadlock.

4. All programs use finite iterations and terminate
normally.

5. Synchronization provides correctness but
introduces
   some execution overhead.
""")
```

```
Philosopher 1 finished eating.
Philosopher 1 is thinking.
Philosopher 5 picked right fork and is eating (Meal 3).
Philosopher 2 is waiting for forks.
Philosopher 2 picked left fork.
Philosopher 2 picked right fork and is eating (Meal 3).Philosopher 3 finished eating.

Philosopher 4 is waiting for forks.
Philosopher 4 picked left fork.
Philosopher 1 is waiting for forks.
Philosopher 4 picked right fork and is eating (Meal 3).
Philosopher 1 picked left fork.
Philosopher 5 finished eating.
Philosopher 2 finished eating.
Philosopher 1 picked right fork and is eating (Meal 3).
Philosopher 4 finished eating.
Philosopher 1 finished eating.

Dining Philosophers completed successfully.

============================================================
FINAL OBSERVATION
============================================================
```

```
2. Readers-Writers:
   - Multiple readers can read simultaneously.
   - Writers get exclusive access.
   - Reader count is protected by a lock.

3. Dining Philosophers:
   - Forks are shared resources.
   - Semaphore limits simultaneous philosophers.
   - This prevents permanent deadlock.

4. All programs use finite iterations and terminate normally.

5. Synchronization provides correctness but introduces
   some execution overhead.
```

```
PS D:\5thSEM KMRU\operting System\LAB WORK\LAB> python -u "d:\5thSEM KMRU\operting System\LAB WORK\LAB\classical_synchronization.py"

============================================================
1. PRODUCER - CONSUMER PROBLEM
============================================================
Producer produced: 1 | Buffer position: 0
Consumer consumed: 1 | Buffer position: 0
Producer produced: 2 | Buffer position: 1
Consumer consumed: 2 | Buffer position: 1
Producer produced: 3 | Buffer position: 2
Producer produced: 4 | Buffer position: 3
Consumer consumed: 3 | Buffer position: 2
Producer produced: 5 | Buffer position: 4
Consumer consumed: 4 | Buffer position: 3
Producer produced: 6 | Buffer position: 0
Producer produced: 7 | Buffer position: 1
Consumer consumed: 5 | Buffer position: 4
Producer produced: 8 | Buffer position: 2
Consumer consumed: 6 | Buffer position: 0
Producer produced: 9 | Buffer position: 3
Producer produced: 10 | Buffer position: 4
Consumer consumed: 7 | Buffer position: 1
Consumer consumed: 8 | Buffer position: 2
Consumer consumed: 9 | Buffer position: 3
Consumer consumed: 10 | Buffer position: 4
Producer-Consumer completed successfully.

============================================================
2. READERS - WRITERS PROBLEM
============================================================
Reader 1 is reading shared data = 0
Reader 2 is reading shared data = 0
Reader 3 is reading shared data = 0
Writer 1 is writing shared data = 1
Writer 2 is writing shared data = 2
Reader 1 is reading shared data = 2Reader 3 is reading shared data = 2Reader 2 is reading shared data = 2


Writer 1 is writing shared data = 3
Writer 2 is writing shared data = 4
```

```
Writer 1 is writing shared data = 3
Writer 2 is writing shared data = 4
Reader 2 is reading shared data = 4Reader 1 is reading shared data = 4

Reader 3 is reading shared data = 4
Writer 1 is writing shared data = 5
Writer 2 is writing shared data = 6
Readers-Writers completed successfully.
```

```
============================================================
3. DINING PHILOSOPHERS PROBLEM
============================================================
Philosopher 1 is thinking.
Philosopher 2 is thinking.
Philosopher 3 is thinking.
Philosopher 4 is thinking.
Philosopher 5 is thinking.
Philosopher 2 is waiting for forks.
Philosopher 2 picked left fork.
Philosopher 2 picked right fork and is eating (Meal 1).
Philosopher 5 is waiting for forks.
Philosopher 5 picked left fork.
Philosopher 5 picked right fork and is eating (Meal 1).
Philosopher 1 is waiting for forks.
Philosopher 4 is waiting for forks.
Philosopher 4 picked left fork.
Philosopher 5 finished eating.Philosopher 1 picked left fork.Philosopher 4 picked right fork and is eating (Meal 1).


Philosopher 3 is waiting for forks.
Philosopher 5 is thinking.
Philosopher 2 finished eating.Philosopher 1 picked right fork and is eating (Meal 1).

Philosopher 2 is thinking.
Philosopher 3 picked left fork.
Philosopher 4 finished eating.Philosopher 3 picked right fork and is eating (Meal 1).

Philosopher 4 is thinking.
Philosopher 5 is waiting for forks.
Philosopher 5 picked left fork.
Philosopher 1 finished eating.Philosopher 5 picked right fork and is eating (Meal 2).
Philosopher 1 is thinking.

Philosopher 2 is waiting for forks.
Philosopher 2 picked left fork.
Philosopher 5 finished eating.
Philosopher 5 is thinking.
Philosopher 4 is waiting for forks.
Philosopher 3 finished eating.Philosopher 4 picked left fork.
```

```
Philosopher 2 picked right fork and is eating (Meal 2).Philosopher 4 picked right fork and is eating (Meal 2).

Philosopher 3 is thinking.
Philosopher 3 is waiting for forks.
Philosopher 1 is waiting for forks.
Philosopher 1 picked left fork.
Philosopher 4 finished eating.
Philosopher 4 is thinking.
Philosopher 3 picked left fork.Philosopher 1 picked right fork and is eating (Meal 2).Philosopher 2 finished eating.


Philosopher 2 is thinking.Philosopher 3 picked right fork and is eating (Meal 2).

Philosopher 5 is waiting for forks.
Philosopher 5 picked left fork.
Philosopher 3 finished eating.
Philosopher 3 is thinking.
Philosopher 1 finished eating.Philosopher 5 picked right fork and is eating (Meal 3).

Philosopher 1 is thinking.
Philosopher 2 is waiting for forks.
Philosopher 2 picked left fork.
Philosopher 2 picked right fork and is eating (Meal 3).
Philosopher 4 is waiting for forks.
Philosopher 4 picked left fork.
Philosopher 3 is waiting for forks.
Philosopher 5 finished eating.Philosopher 4 picked right fork and is eating (Meal 3).

Philosopher 2 finished eating.
Philosopher 3 picked left fork.
Philosopher 1 is waiting for forks.
Philosopher 1 picked left fork.
Philosopher 1 picked right fork and is eating (Meal 3).
Philosopher 4 finished eating.
Philosopher 3 picked right fork and is eating (Meal 3).
Philosopher 1 finished eating.
Philosopher 3 finished eating.

Dining Philosophers completed successfully.

============================================================
```

```
============================================================
FINAL OBSERVATION
============================================================

1. Producer-Consumer:
   - Semaphore controls empty and full buffer slots.
   - Mutex provides exclusive buffer access.

2. Readers-Writers:
   - Multiple readers can read simultaneously.
   - Writers get exclusive access.
   - Reader count is protected by a lock.

3. Dining Philosophers:
   - Forks are shared resources.
   - Semaphore limits simultaneous philosophers.
   - This prevents permanent deadlock.

4. All programs use finite iterations and terminate normally.

5. Synchronization provides correctness but introduces
   some execution overhead.

PS D:\5thSEM KMRU\operting System\LAB WORK\LAB>
```